PENJELASAN PROGRAM

Aplikasi Kantin Kampus dengan Cubit dan BLoC

Modul V – State Management pada Aplikasi Flutter • Pemrograman Perangkat Bergerak Lanjut

1. Gambaran Umum Program

Program ini adalah aplikasi **Kantin Kampus** yang dibuat untuk mengerjakan bagian **E. Materi Praktikum** pada Modul 5. Pengguna dapat memilih menu makanan/minuman, memasukkannya ke keranjang, mengubah jumlahnya, melihat total harga, dan mengosongkan keranjang.

Aplikasi yang sama dibuat dalam **dua versi state management**, yaitu **Cubit** dan **BLoC**, keduanya dari paket `flutter_bloc`. Agar mudah dibandingkan, kedua versi diletakkan dalam satu proyek (`progamsederhana`). Saat aplikasi dibuka, pengguna memilih tombol **Versi Cubit** atau **Versi BLoC**. Tampilan kedua versi sama persis; yang berbeda hanya cara mengelola state di belakangnya.

1.1 Struktur Folder

File	Fungsi
lib/models/produk.dart	Kelas data Produk dan ItemKeranjang, daftar menu, serta fungsi bantu (format rupiah, hitung total).
lib/cubit/keranjang_cubit.dart	Logika keranjang versi Cubit (method tambah, kurangi, kosongkan).
lib/bloc/keranjang_bloc.dart	Logika keranjang versi BLoC (Event, State, dan Bloc).
lib/pages/versi_cubit.dart	Tampilan halaman Produk dan Keranjang yang memakai KeranjangCubit.
lib/pages/versi_bloc.dart	Tampilan yang sama memakai KeranjangBloc, ditambah SnackBar lewat BlocListener.
lib/main.dart	Titik awal aplikasi dan halaman pilih versi; tempat BlocProvider dipasang.

1.2 Alur Data

Cubit: tombol ditekan → widget memanggil method Cubit (misalnya `tambah(produk)`) → Cubit membuat daftar baru lalu memanggil `emit()` → `BlocBuilder` menggambar ulang tampilan.

BLoC: tombol ditekan → widget mengirim Event dengan `add(TambahProduk(produk))` → Bloc menangani Event di handler `on<TambahProduk>` → Bloc memanggil `emit()` dengan State baru → `BlocBuilder` menggambar ulang tampilan.

2. Penjelasan Program

Setiap bagian berisi penjelasan, **Screenshot Program** (kode di editor, lengkap dengan nomor baris), dan **Screenshot Hasil** (tampilan aplikasi yang dihasilkan kode tersebut, ukuran layar ponsel 360 × 780).

2.1 Model Data – lib/models/produk.dart

File ini berisi data yang dipakai bersama oleh versi Cubit dan BLoC:

- **Produk** menyimpan nama, harga, dan ikon (emoji) satu menu.
- **ItemKeranjang** menyimpan satu Produk beserta jumlahnya di keranjang. Nilai awal jumlah adalah 1.
- **daftarProduk** adalah tiga menu contoh: Nasi Goreng, Mie Ayam, dan Es Teh.
- Fungsi bantu `rupiah()`, `hitungTotalHarga()`, dan `hitungTotalItem()` dibuat sekali di sini supaya tidak ditulis berulang di setiap halaman.

Semua atribut memakai `final` dan konstruktornya `const`, sehingga objek tidak bisa diubah setelah dibuat (immutable). Untuk mengubah jumlah, program membuat objek ItemKeranjang yang baru.

Screenshot Program

Dart lib/models/produk.dart

```dart
// Model data: dipakai bersama oleh versi Cubit dan versi BLoC.

class Produk {
  const Produk({required this.nama, required this.harga, required this.ikon});

  final String nama;
  final int harga;
  final String ikon;
}

class ItemKeranjang {
  const ItemKeranjang({required this.produk, this.jumlah = 1});

  final Produk produk;
  final int jumlah;
}

// Daftar menu kantin (data contoh).
const daftarProduk = [
  Produk(nama: 'Nasi Goreng', harga: 15000, ikon: '🍛'),
  Produk(nama: 'Mie Ayam', harga: 13000, ikon: '🍜'),
  Produk(nama: 'Es Teh', harga: 5000, ikon: '🧋'),
];

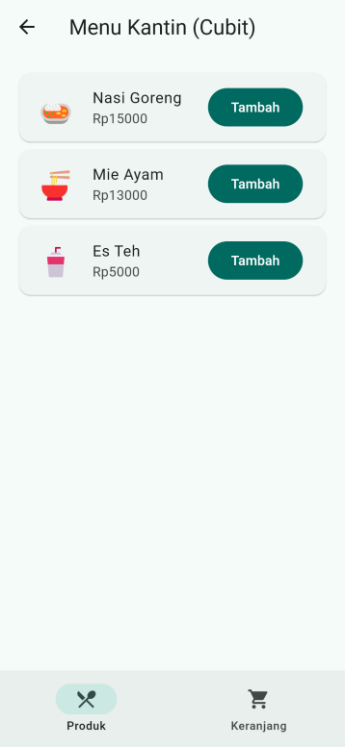
// Fungsi bantu supaya format harga sama di semua halaman.
String rupiah(int nilai) => 'Rp$nilai';

int hitungTotalHarga(List<ItemKeranjang> items) =>
    items.fold(0, (total, item) => total + item.produk.harga * item.jumlah);

int hitungTotalItem(List<ItemKeranjang> items) =>
    items.fold(0, (total, item) => total + item.jumlah);
```

Gambar 1. Kode lib/models/produk.dart (baris 1-32)

Screenshot Hasil



Gambar 2. Halaman Produk (Cubit)

Daftar menu dari daftarProduk; tombol Tambah memanggil cubit.tambah().

2.2 KeranjangCubit – lib/cubit/keranjang_cubit.dart

`KeranjangCubit` meng-extend `Cubit<List<ItemKeranjang>>`, artinya state-nya langsung berupa daftar item keranjang. Nilai awalnya daftar kosong (`super([])`).

- **tambah(produk)**: menyalin daftar lama dengan `[...state]`, mencari produk dengan `indexWhere`. Jika belum ada (indeks -1), produk ditambahkan dengan jumlah 1. Jika sudah ada, item diganti dengan jumlah + 1.
- **kurangi(produk)**: jika jumlahnya tinggal 1, item dihapus dari keranjang. Jika lebih dari 1, jumlahnya dikurangi 1.
- **kosongkan()**: langsung mengirim daftar kosong.

Setiap method diakhiri `emit(daftar)`. Daftar lama **tidak diubah langsung**; selalu dibuat salinan baru. Dengan begitu flutter_bloc tahu bahwa state berubah dan widget yang mendengarkan akan digambar ulang. Ini perbedaan utama dengan Provider yang mengubah data langsung lalu memanggil `notifyListeners()`.

Screenshot Program

Dart lib/cubit/keranjang_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

import '../models/produk.dart';

// CUBIT: state-nya langsung berupa List<ItemKeranjang>.
// Widget memanggil method (tambah/kurangi/kosongkan), lalu Cubit memanggil emit().
class KeranjangCubit extends Cubit<List<ItemKeranjang>> {
  KeranjangCubit() : super([]); // state awal: keranjang kosong

  void tambah(Produk produk) {
    final daftar = [...state]; // salin daftar lama (jangan diubah langsung)
    final i = daftar.indexWhere((item) => item.produk.nama == produk.nama);

    if (i == -1) {
      daftar.add(ItemKeranjang(produk: produk)); // produk baru
    } else {
      daftar[i] = ItemKeranjang(produk: produk, jumlah: daftar[i].jumlah + 1);
    }
    emit(daftar); // kirim state baru -> BlocBuilder menggambar ulang
  }

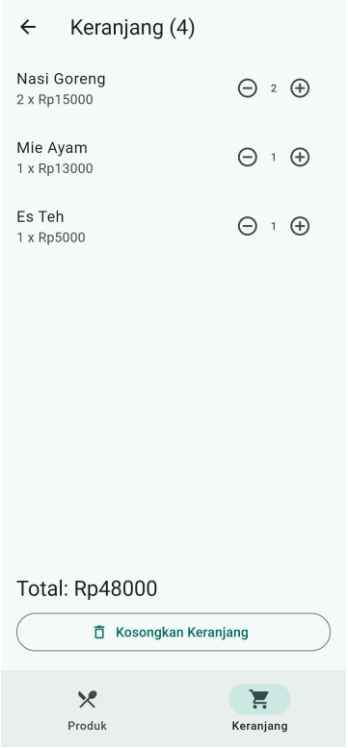
  void kurangi(Produk produk) {
    final daftar = [...state];
    final i = daftar.indexWhere((item) => item.produk.nama == produk.nama);
    if (i == -1) return;

    if (daftar[i].jumlah == 1) {
      daftar.removeAt(i); // jumlah tinggal 1 -> hapus dari keranjang
    } else {
      daftar[i] = ItemKeranjang(produk: produk, jumlah: daftar[i].jumlah - 1);
    }
    emit(daftar);
  }

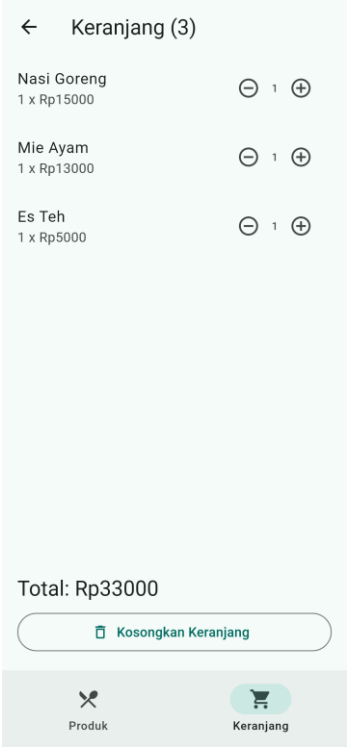
  void kosongkan() => emit([]);
}
```

Gambar 3. Kode lib/cubit/keranjang_cubit.dart (baris 1-36)

Screenshot Hasil



Gambar 4. Keranjang terisi (Cubit)
Nasi Goreng 2×, Mie Ayam dan Es Teh 1×. Judul (4), total Rp48000.



Gambar 5. Setelah tombol – ditekan
Nasi Goreng menjadi 1. Judul (3) dan total Rp33000 ikut berubah.



Gambar 6. Keranjang dikosongkan
kosongkan() → emit([]), BlocBuilder menampilkan pesan kosong.

2.3 KeranjangBloc – lib/bloc/keranjang_bloc.dart

Versi BLoC membutuhkan **tiga bagian**:

- **Event** (`KeranjangEvent`) menggambarkan "apa yang terjadi". Ada tiga turunan: `TambahProduk` dan `KurangiProduk` (membawa data Produk), serta `KosongkanKeranjang` (tanpa data).
- **State** (`KeranjangState`) menggambarkan "apa yang ditampilkan", yaitu objek yang membungkus `List<ItemKeranjang> items`.
- **Bloc** (`KeranjangBloc`) mendaftarkan handler `on<NamaEvent>` di konstruktornya. Setiap handler menerima event, menjalankan logika, lalu memanggil `emit(KeranjangState(...))`.

Isi setiap handler **sama persis** dengan method pada KeranjangCubit. Perbedaannya hanya pada cara aksi dikirim: Cubit dipanggil langsung lewat method, sedangkan BLoC harus lewat objek Event. Karena setiap aksi berupa objek, riwayat perubahan lebih mudah dilacak dan diuji.

Screenshot Program

Dart lib/bloc/keranjang_bloc.dart (bagian 1 dari 2)

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

import '../models/produk.dart';

// ===== 1. EVENT: "apa yang terjadi" (dikirim dari tampilan lewat add()) =====
abstract class KeranjangEvent {}

class TambahProduk extends KeranjangEvent {
  TambahProduk(this.produk);
  final Produk produk;
}

class KurangiProduk extends KeranjangEvent {
  KurangiProduk(this.produk);
  final Produk produk;
}

class KosongkanKeranjang extends KeranjangEvent {}

// ===== 2. STATE: "apa yang ditampilkan" di layar =====
class KeranjangState {
  const KeranjangState(this.items);
  final List<ItemKeranjang> items;
}

// ===== 3. BLOC: menerima Event -> menjalankan logika -> emit State baru =====
class KeranjangBloc extends Bloc<KeranjangEvent, KeranjangState> {
  KeranjangBloc() : super(const KeranjangState([])) {
    // Isi tiap handler sama persis dengan method di KeranjangCubit.
    on<TambahProduk>((event, emit) {
      final daftar = [...state.items];
      final i = daftar.indexWhere((item) => item.produk.nama == event.produk.nama);

      if (i == -1) {
        daftar.add(ItemKeranjang(produk: event.produk));
      } else {
        daftar[i] = ItemKeranjang(produk: event.produk, jumlah: daftar[i].jumlah + 1);
      }
      emit(KeranjangState(daftar));
    });

```

Gambar 7. Kode lib/bloc/keranjang_bloc.dart (baris 1-41)

Dart lib/bloc/keranjang_bloc.dart (bagian 2 dari 2)

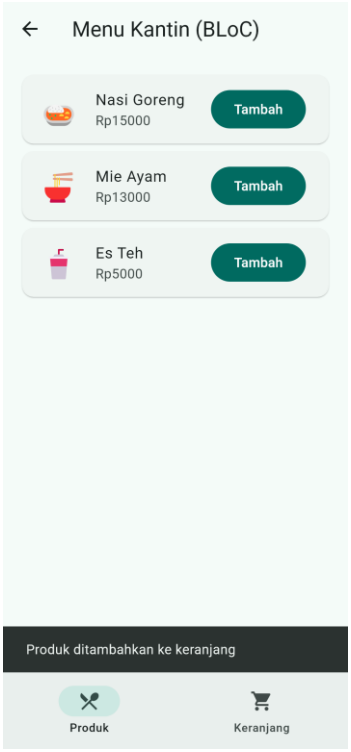
```dart
    on<KurangiProduk>((event, emit) {
      final daftar = [...state.items];
      final i = daftar.indexWhere((item) => item.produk.nama == event.produk.nama);
      if (i == -1) return;

      if (daftar[i].jumlah == 1) {
        daftar.removeAt(i);
      } else {
        daftar[i] = ItemKeranjang(produk: event.produk, jumlah: daftar[i].jumlah - 1);
      }
      emit(KeranjangState(daftar));
    });

    on<KosongkanKeranjang>((event, emit) => emit(const KeranjangState([])));
  }
}
```

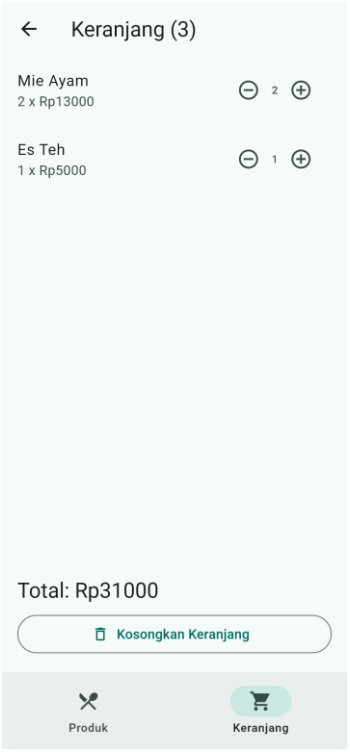
Gambar 8. Kode lib/bloc/keranjang_bloc.dart (baris 42-57)

Screenshot Hasil



Gambar 9. SnackBar versi BLoC

Event TambahProduk dikirim; BlocListener memunculkan SnackBar sekali.



Gambar 10. Keranjang terisi (BLoC)

Mie Ayam 2× dan Es Teh 1×. Hasil identik dengan versi Cubit.

2.4 Tampilan Versi Cubit – lib/pages/versi_cubit.dart

File ini berisi tiga widget:

- **KantinCubit** adalah kerangka halaman dengan `AppBar`, `IndexedStack`, dan `NavigationBar` (Produk dan Keranjang). Indeks tab yang aktif disimpan dengan `setState()` karena hanya penting untuk widget ini sendiri (state lokal). Jumlah item di judul AppBar dibaca dengan `context.select`, sehingga widget hanya dibangun ulang jika jumlah item berubah.
- **HalamanProdukCubit** menampilkan daftar menu dalam `Card`. Tombol **Tambah** memanggil `context.read<KeranjangCubit>().tambah(produk)`. Di sini dipakai `context.read` (bukan watch) karena tombol hanya perlu mengirim aksi, tidak perlu ikut digambar ulang.
- **HalamanKeranjangCubit** dibungkus `BlocBuilder<KeranjangCubit, List<ItemKeranjang>>`. Setiap kali Cubit memanggil `emit()`, builder dijalankan ulang: menampilkan tulisan "Keranjang masih kosong" jika kosong, atau daftar item dengan tombol −/+, total harga, dan tombol **Kosongkan Keranjang**.

`IndexedStack` membuat kedua halaman tetap hidup di belakang layar, sehingga posisi gulir tidak hilang saat berpindah tab.

Screenshot Program

Dart lib/pages/versi_cubit.dart (bagian 1 dari 3)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/keranjang_cubit.dart';
import '../models/produk.dart';

// Kerangka aplikasi versi CUBIT: NavigationBar Produk <-> Keranjang.
class KantinCubit extends StatefulWidget {
  const KantinCubit({super.key});

  @override
  State<KantinCubit> createState() => _KantinCubitState();
}

class _KantinCubitState extends State<KantinCubit> {
  int _indeks = 0; // state lokal -> cukup pakai setState()

  @override
  Widget build(BuildContext context) {
    // context.select: hanya rebuild kalau totalItem berubah.
    final totalItem = context.select<KeranjangCubit, int>(
      (cubit) => hitungTotalItem(cubit.state),
    );

    return Scaffold(
      appBar: AppBar(
        title: Text(_indeks == 0 ? 'Menu Kantin (Cubit)' : 'Keranjang ($totalItem)'),
      ),
      body: IndexedStack(
        index: _indeks,
        children: const [HalamanProdukCubit(), HalamanKeranjangCubit()],
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeks,
        onDestinationSelected: (i) => setState(() => _indeks = i),
        destinations: const [
          NavigationDestination(icon: Icon(Icons.restaurant_menu), label: 'Produk'),
          NavigationDestination(icon: Icon(Icons.shopping_cart), label: 'Keranjang'),
        ],
      ),
    );
  }
}

```

Gambar 11. Kode lib/pages/versi_cubit.dart (baris 1-44)

Dart lib/pages/versi_cubit.dart (bagian 2 dari 3)

```dart
// ---------------- Halaman Produk ----------------
class HalamanProdukCubit extends StatelessWidget {
  const HalamanProdukCubit({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(16),
      children: [
        for (final produk in daftarProduk)
          Card(
            child: ListTile(
              leading: Text(produk.ikon, style: const TextStyle(fontSize: 32)),
              title: Text(produk.nama),
              subtitle: Text(rupiah(produk.harga)),
              trailing: FilledButton(
                // CUBIT: panggil method secara langsung
                onPressed: () => context.read<KeranjangCubit>().tambah(produk),
                child: const Text('Tambah'),
              ),
            ),
          ),
      ],
    );
  }
}

// ---------------- Halaman Keranjang ----------------
class HalamanKeranjangCubit extends StatelessWidget {
  const HalamanKeranjangCubit({super.key});

  @override
  Widget build(BuildContext context) {
    // BlocBuilder: menggambar ulang setiap kali Cubit emit state baru.
    return BlocBuilder<KeranjangCubit, List<ItemKeranjang>>(
      builder: (context, items) {
        if (items.isEmpty) {
          return const Center(child: Text('Keranjang masih kosong'));
        }

```

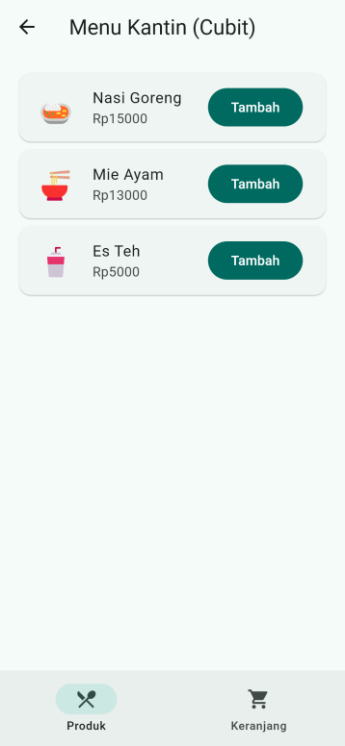
Gambar 12. Kode lib/pages/versi_cubit.dart (baris 45-84)

Dart lib/pages/versi_cubit.dart (bagian 3 dari 3)

```dart
        final cubit = context.read<KeranjangCubit>();
        return Column(
          children: [
            Expanded(
              child: ListView(
                children: [
                  for (final item in items)
                    ListTile(
                      title: Text(item.produk.nama),
                      subtitle: Text('${item.jumlah} x ${rupiah(item.produk.harga)}'),
                      trailing: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          IconButton(
                            onPressed: () => cubit.kurangi(item.produk),
                            icon: const Icon(Icons.remove_circle_outline),
                          ),
                          Text('${item.jumlah}'),
                          IconButton(
                            onPressed: () => cubit.tambah(item.produk),
                            icon: const Icon(Icons.add_circle_outline),
                          ),
                        ],
                      ),
                    ),
                ],
              ),
            ),
            Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Text(
                    'Total: ${rupiah(hitungTotalHarga(items))}',
                    style: Theme.of(context).textTheme.titleLarge,
                  ),
                  const SizedBox(height: 8),
                  OutlinedButton.icon(
                    onPressed: cubit.kosongkan,
                    icon: const Icon(Icons.delete_outline),
                    label: const Text('Kosongkan Keranjang'),
                  ),
                ],
              ),
            ),
          ],
        );
      },
    );
  }
}
```

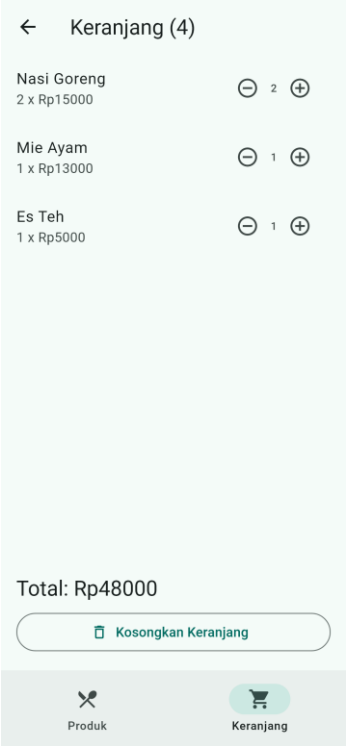
Gambar 13. Kode lib/pages/versi_cubit.dart (baris 85-136)

Screenshot Hasil



Gambar 14. Halaman Produk (Cubit)

Daftar menu dari daftarProduk; tombol Tambah memanggil cubit.tambah().



Gambar 15. Keranjang terisi (Cubit)

Nasi Goreng 2×, Mie Ayam dan Es Teh 1×. Judul (4), total Rp48000.



Gambar 16. Keranjang dikosongkan

kosongkan() → emit([]), BlocBuilder menampilkan pesan kosong.

2.5 Tampilan Versi BLoC + SnackBar – lib/pages/versi_bloc.dart

Strukturnya sama dengan versi Cubit. Perbedaannya:

- Semua tombol mengirim Event, misalnya `context.read<KeranjangBloc>().add(TambahProduk(produk))`.
- `BlocBuilder<KeranjangBloc, KeranjangState>` membaca data dari `state.items`.
- **HalamanProdukBloc** dibungkus `BlocListener` untuk menampilkan **SnackBar** "Produk ditambahkan ke keranjang". SnackBar adalah **efek samping** (cukup muncul sekali), sehingga ditaruh di `listener`, bukan di `builder`. Jika ditaruh di builder, SnackBar bisa muncul berulang setiap kali tampilan digambar ulang.
- `listenWhen` membatasi listener agar hanya berjalan ketika jumlah item **bertambah**. Tanpa ini, SnackBar juga muncul saat item dikurangi.
- `clearSnackBars()` membuang SnackBar lama (termasuk yang masih antre) sebelum menampilkan yang baru, dan juga dipanggil saat berpindah tab, agar SnackBar tidak menumpuk atau menutupi tombol Kosongkan Keranjang.

Screenshot Program

Dart lib/pages/versi_bloc.dart (bagian 1 dari 4)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

// Kerangka aplikasi versi BLOC: sama persis dengan versi Cubit,
// bedanya hanya sumber data (KeranjangBloc) dan cara mengirim aksi (add Event).
class KantinBloc extends StatefulWidget {
  const KantinBloc({super.key});

  @override
  State<KantinBloc> createState() => _KantinBlocState();
}

class _KantinBlocState extends State<KantinBloc> {
  int _indeks = 0;

  @override
  Widget build(BuildContext context) {
    final totalItem = context.select<KeranjangBloc, int>(
      (bloc) => hitungTotalItem(bloc.state.items),
    );

    return Scaffold(
      appBar: AppBar(
        title: Text(_indeks == 0 ? 'Menu Kantin (BLoC)' : 'Keranjang ($totalItem)'),
      ),
      body: IndexedStack(
        index: _indeks,
        children: const [HalamanProdukBloc(), HalamanKeranjangBloc()],
      ),
      bottomNavigationBar: NavigationBar(
        selectedIndex: _indeks,
        onDestinationSelected: (i) {
          // Tutup SnackBar agar tidak menutupi tombol di halaman lain.
          ScaffoldMessenger.of(context).clearSnackBars();
          setState(() => _indeks = i);
        },
        destinations: const [
          NavigationDestination(icon: Icon(Icons.restaurant_menu), label: 'Produk'),
          NavigationDestination(icon: Icon(Icons.shopping_cart), label: 'Keranjang'),
        ],
      ),
    );
```

Gambar 17. Kode lib/pages/versi_bloc.dart (baris 1-45)

Dart lib/pages/versi_bloc.dart (bagian 2 dari 4)

```dart
  }
}

// ---------------- Halaman Produk (+ SnackBar lewat BlocListener) ----------------
class HalamanProdukBloc extends StatelessWidget {
  const HalamanProdukBloc({super.key});

  @override
  Widget build(BuildContext context) {
    // BlocListener: untuk EFEK SAMPING (SnackBar), tidak menggambar apa pun.
    return BlocListener<KeranjangBloc, KeranjangState>(
      // Hanya bereaksi saat jumlah item BERTAMBAH (bukan saat dikurangi/dikosongkan).
      listenWhen: (lama, baru) =>
          hitungTotalItem(baru.items) > hitungTotalItem(lama.items),
      listener: (context, state) {
        ScaffoldMessenger.of(context)
          ..clearSnackBars() // buang SnackBar lama agar tidak menumpuk
          ..showSnackBar(
            const SnackBar(content: Text('Produk ditambahkan ke keranjang')),
          );
      },
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          for (final produk in daftarProduk)
            Card(
              child: ListTile(
                leading: Text(produk.ikon, style: const TextStyle(fontSize: 32)),
                title: Text(produk.nama),
                subtitle: Text(rupiah(produk.harga)),
                trailing: FilledButton(
                  // BLOC: kirim Event lewat add()
                  onPressed: () =>
                      context.read<KeranjangBloc>().add(TambahProduk(produk)),
                  child: const Text('Tambah'),
                ),
              ),
            ),
        ],
      ),
    );
  }
}

```

Gambar 18. Kode lib/pages/versi_bloc.dart (baris 46-89)

Dart lib/pages/versi_bloc.dart (bagian 3 dari 4)

```dart
// ---------------- Halaman Keranjang ----------------
class HalamanKeranjangBloc extends StatelessWidget {
  const HalamanKeranjangBloc({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<KeranjangBloc, KeranjangState>(
      builder: (context, state) {
        final items = state.items;
        if (items.isEmpty) {
          return const Center(child: Text('Keranjang masih kosong'));
        }

        final bloc = context.read<KeranjangBloc>();
        return Column(
          children: [
            Expanded(
              child: ListView(
                children: [
                  for (final item in items)
                    ListTile(
                      title: Text(item.produk.nama),
                      subtitle: Text('${item.jumlah} x ${rupiah(item.produk.harga)}'),
                      trailing: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          IconButton(
                            onPressed: () => bloc.add(KurangiProduk(item.produk)),
                            icon: const Icon(Icons.remove_circle_outline),
                          ),
                          Text('${item.jumlah}'),
                          IconButton(
                            onPressed: () => bloc.add(TambahProduk(item.produk)),
                            icon: const Icon(Icons.add_circle_outline),
                          ),
                        ],
                      ),
                    ),
                ],
              ),
            ),
            Padding(
              padding: const EdgeInsets.all(16),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
```

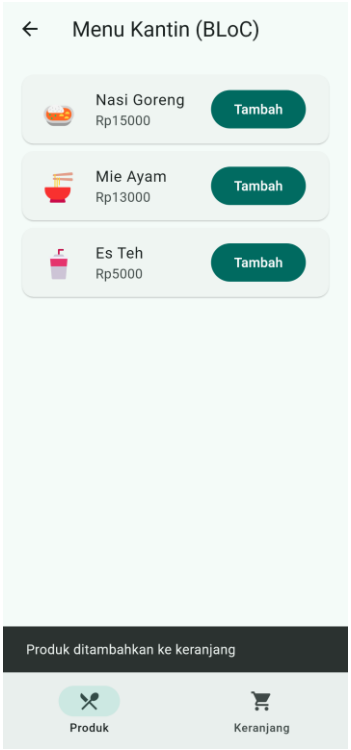
Gambar 19. Kode lib/pages/versi_bloc.dart (baris 90-134)

Dart lib/pages/versi_bloc.dart (bagian 4 dari 4)

```dart
                children: [
                  Text(
                    'Total: ${rupiah(hitungTotalHarga(items))}',
                    style: Theme.of(context).textTheme.titleLarge,
                  ),
                  const SizedBox(height: 8),
                  OutlinedButton.icon(
                    onPressed: () => bloc.add(KosongkanKeranjang()),
                    icon: const Icon(Icons.delete_outline),
                    label: const Text('Kosongkan Keranjang'),
                  ),
                ],
              ),
            ),
          ],
        );
      },
    );
  }
}
```

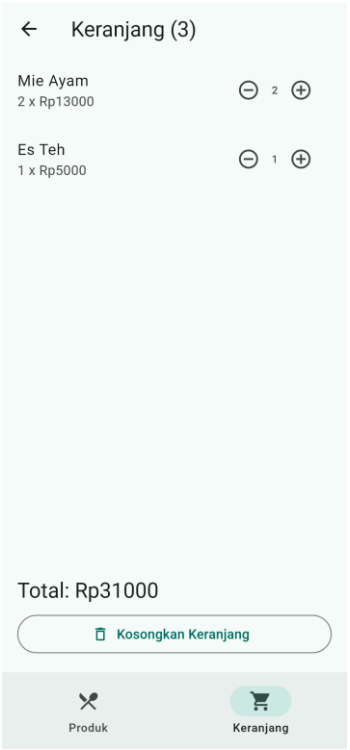
Gambar 20. Kode lib/pages/versi_bloc.dart (baris 135-154)

Screenshot Hasil



Gambar 21. SnackBar versi BLoC

Event TambahProduk dikirim; BlocListener memunculkan SnackBar sekali.



Gambar 22. Keranjang terisi (BLoC)

Mie Ayam 2× dan Es Teh 1×. Hasil identik dengan versi Cubit.

2.6 Titik Awal Aplikasi – lib/main.dart

`main()` menjalankan `AplikasiKantin`, yaitu `MaterialApp` bertema Material 3 warna teal. Halaman awalnya `HalamanPilihVersi` dengan dua tombol.

Setiap tombol membuka halaman baru yang dibungkus **BlocProvider**. BlocProvider membuat objek Cubit/Bloc (`create: (_) => KeranjangCubit()`) dan menyediakannya untuk semua widget di bawahnya, sehingga `context.read` dan `BlocBuilder` di halaman Produk maupun Keranjang dapat menemukannya. BlocProvider juga otomatis menutup Cubit/Bloc saat halaman ditutup, dan keranjang kembali kosong ketika versi dibuka ulang.

Screenshot Program

Dart lib/main.dart (bagian 1 dari 2)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/keranjang_bloc.dart';
import 'cubit/keranjang_cubit.dart';
import 'pages/versi_bloc.dart';
import 'pages/versi_cubit.dart';

void main() => runApp(const AplikasiKantin());

class AplikasiKantin extends StatelessWidget {
  const AplikasiKantin({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Kantin Kampus',
      theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
      home: const HalamanPilihVersi(),
    );
  }
}

// Halaman awal: pilih mau mencoba versi Cubit atau versi BLoC.
class HalamanPilihVersi extends StatelessWidget {
  const HalamanPilihVersi({super.key});

  void _buka(BuildContext context, Widget halaman) {
    Navigator.push(context, MaterialPageRoute(builder: (_) => halaman));
  }

```

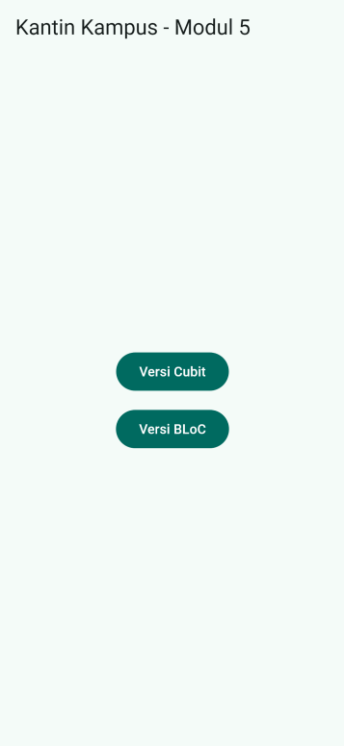
Gambar 23. Kode lib/main.dart (baris 1-32)

Dart lib/main.dart (bagian 2 dari 2)

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Kantin Kampus - Modul 5')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            FilledButton(
              // BlocProvider menyediakan KeranjangCubit untuk semua widget di bawahnya.
              onPressed: () => _buka(
                context,
                BlocProvider(
                  create: (_) => KeranjangCubit(),
                  child: const KantinCubit(),
                ),
              ),
              child: const Text('Versi Cubit'),
            ),
            const SizedBox(height: 12),
            FilledButton(
              onPressed: () => _buka(
                context,
                BlocProvider(
                  create: (_) => KeranjangBloc(),
                  child: const KantinBloc(),
                ),
              ),
              child: const Text('Versi BLoC'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Gambar 24. Kode lib/main.dart (baris 33-68)

Screenshot Hasil



Gambar 25. Halaman pilih versi

Tampilan awal: memilih Versi Cubit atau Versi BLoC.

3. Perbandingan Provider, Cubit, dan BLoC

Aspek	Provider (Modul 3)	Cubit	BLoC
Kelas yang dibuat	1 (KeranjangModel)	1 (KeranjangCubit)	3 (Event, State, Bloc)
Bentuk state	Map di dalam kelas	List<ItemKeranjang>	KeranjangState berisi items
Cara mengubah data	Ubah langsung, lalu notifyListeners()	Buat daftar baru, lalu emit()	add(Event), ditangani on<Event>, lalu emit()
Aksi tombol Tambah	model.tambah(p)	cubit.tambah(p)	bloc.add(TambahProduk(p))
Cara membaca data	Consumer	BlocBuilder	BlocBuilder
Widget penyedia	ChangeNotifierProvider	BlocProvider	BlocProvider
Cocok untuk	State sederhana yang dibagikan	Logika sederhana dan langsung	Logika kompleks, banyak sumber aksi

4. Pengujian

Program diuji dengan `flutter analyze` (hasil: **No issues found**) dan `flutter test` (hasil: **All tests passed**, 4 pengujian):

- Unit test KeranjangCubit: tambah 2×, kurangi 1×, lalu kosongkan.
- Unit test KeranjangBloc: mengirim Event TambahProduk, KurangiProduk, dan KosongkanKeranjang, lalu memeriksa State.
- Widget test versi Cubit dan versi BLoC: menambah produk, memeriksa judul "Keranjang (2)" dan total "Rp20000", lalu mengosongkan keranjang.

5. Kesimpulan

Logika keranjang pada Cubit dan BLoC sama persis. Yang berbeda adalah **cara aksi dikirim**: Cubit cukup memanggil method, sedangkan BLoC harus mengirim Event. Cubit lebih ringkas sehingga cocok untuk logika sederhana seperti keranjang ini. BLoC memerlukan kode lebih banyak, tetapi setiap aksi tercatat sebagai Event, sehingga lebih mudah dilacak dan diuji pada aplikasi yang lebih besar.

`BlocBuilder` dipakai untuk menampilkan data, `BlocListener` untuk efek samping seperti SnackBar, `context.read` untuk mengirim aksi dari tombol, dan `setState()` tetap dipakai untuk state lokal seperti indeks tab.